

Résolution d'un problème inverse: déconvolution d'un signal triangulaire

KEBBAL Hawa

04/02/2025

```
clc; clear; close all;
```

1. Définition des paramètres

```
L = 100; % Durée d'observation du signal d'entrée (secondes)
fe = 1; % Fréquence d'échantillonnage (Hz)
Te = 1/fe; % Période d'échantillonnage (secondes)
Nx = L/Te; % Nombre d'échantillons du signal x
nx = 0:Nx-1; % Indice temporel (entiers)
tx = nx * Te; % Base de temps (secondes)
```

a. Simulation du signal d'entrée

Définition du signal triangulaire sur $[0, L]$

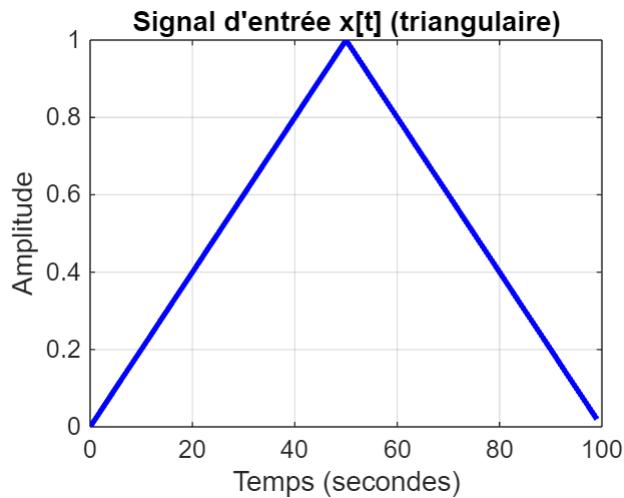
```
x1 = (tx(tx <= (L / 2))) / (L / 2); % Partie croissante [0, L/2]
x2 = (-tx(tx > (L / 2)) + L) / (L / 2); % Partie décroissante [L/2, L]

% Assemblage du signal triangulaire complet
x = [x1 x2];
```

Le signal d'entrée est un signal triangulaire qui varie de manière linéaire entre 0 et 1, puis décroît de 1 à 0. Cette forme est choisie car elle permet d'analyser les effets du filtre gaussien et de la quantification.

Affichage du signal $x[t]$

```
%a. Simulation du signal d'entrée
figure;
set(gcf, 'Position', [100, 100, 800, 600]);
plot(tx, x, 'b', 'LineWidth', 2);
title('Signal d\'entrée x[t] (triangulaire)');
xlabel('Temps (secondes)');
ylabel('Amplitude');
grid on;
xlim([0 L]);
```



b. Simulation du filtre

Définition des paramètres du filtre gaussien

```
m = 15;      % Moyenne
sigma = 4;   % Écart-type racine de la variance
Nh = 30;     % Horizon de temps de la réponse impulsionnelle (secondes)
nh = 0:Nh-1; % Indices temporels
```

Stockercette réponse impulsionnelle dans un vecteur et la représenter.

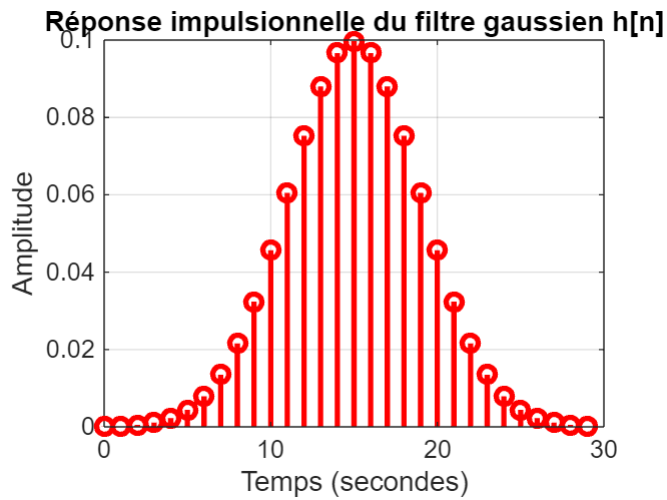
b.1. Discrétisation de la réponse impulsionnelle $h[n]$

```
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 * sigma^2));
```

La réponse impulsionnelle du filtre gaussien montre une courbe en cloche caractéristique. Ce type de filtre est souvent utilisé en traitement du signal pour adoucir les variations brusques du signal d'entrée.

b.2. Affichage de la réponse impulsionnelle

```
figure;
stem(nh, h, 'r', 'LineWidth', 2);
title('Réponse impulsionnelle du filtre gaussien h[n]');
xlabel('Temps (secondes)');
ylabel('Amplitude');
grid on;
xlim([0 Nh]);
```



c. Simulation du problème direct: première approche

C.5.Simulation du signal de sortie non bruité $y_{nb}[n]$ à l'aide de la fonction conv.

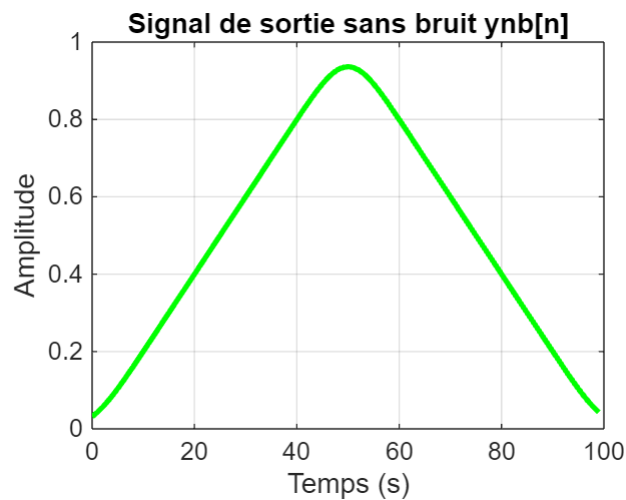
```

y_{nb} = conv(x, h, 'same'); % Convolution discrète sans bruit

% Affichage du signal de sortie non bruité

figure;
plot(tx, y_{nb}, 'g', 'LineWidth', 2);
title('Signal de sortie sans bruit y_{nb}[n]');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

```



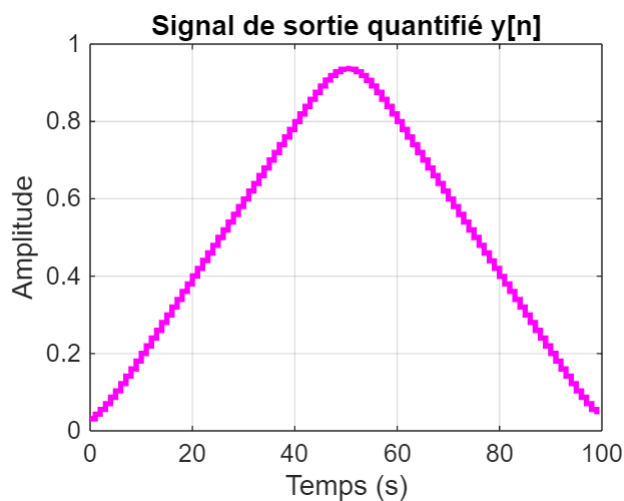
Après avoir convolué le signal d'entrée avec la réponse impulsionnelle du filtre, nous obtenons un signal de sortie qui est une version lissée du signal d'entrée. Ce signal conserve la forme générale du signal triangulaire, mais les transitions deviennent moins abruptes en raison de l'effet du filtre.

C.6. Simulation du signal quantifié et sa représentation à l'aide de la fonction stairs.

```
y = round(y nb * 2^10) / 2^10; % Quantification

% Affichage du signal quantifié

figure;
stairs(tx, y, 'm', 'LineWidth', 2);
title('Signal de sortie quantifié y[n]');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```



C.7. Détermination et représentation de bruit de quantification b[n].

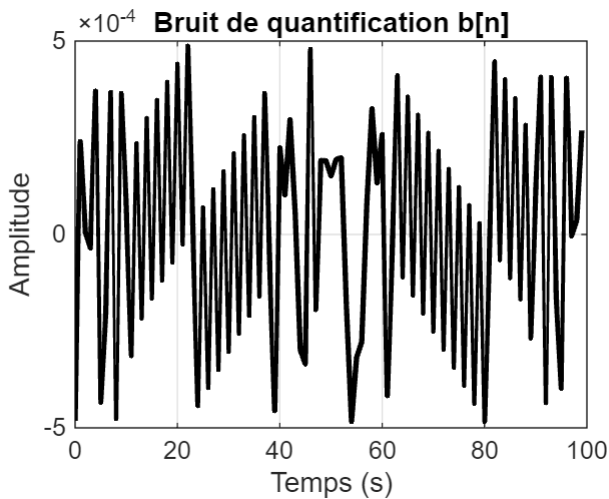
Le bruit de quantification est la différence entre le signal quantifié et le signal non bruité. Ce bruit apparaît comme une variation aléatoire dans le signal quantifié,

```
bquant = y - y nb; % Bruit de quantification réel

% Affichage du bruit de quantification

figure;
plot(tx, bquant, 'k', 'LineWidth', 2);
title('Bruit de quantification b[n]');
xlabel('Temps (s)');
```

```
ylabel('Amplitude');
grid on;
```



d. Simulation du problème direct par approche matricielle

Cette section utilise une approche matricielle pour simuler la convolution du signal triangulaire avec le filtre gaussien en utilisant la multiplication matricielle $y_{nb} = Hx$.

d.8. Détermination de la dimension N_y du vecteur de sortie y_{nb}

```
% Taille des signaux
Nx = length(x);
Nh = length(h);
```

d.9. Construction de la matrice de convolution H avec `toeplitz`

Initialisation du premier vecteur colonne et ligne de H

```
% Taille des signaux
Nx = length(x);
Nh = length(h);
decalage = -20; % Mettre une valeur NÉGATIVE pour décaler à gauche

% Décalage négatif = supprimer les premières valeurs de h
h_modifie = h(abs(decalage)+1:end); % Tronquer les premières valeurs

% Construction de la matrice de convolution H
colonne_H = [h_modifie(:); zeros(Nx - length(h_modifie), 1)];
ligne_H = [h_modifie(1), zeros(1, Nx - 1)];

% Construction de la matrice Toeplitz
H = toeplitz(colonne_H, ligne_H);
```

10. Calcul de la sortie non bruitée par approche matricielle

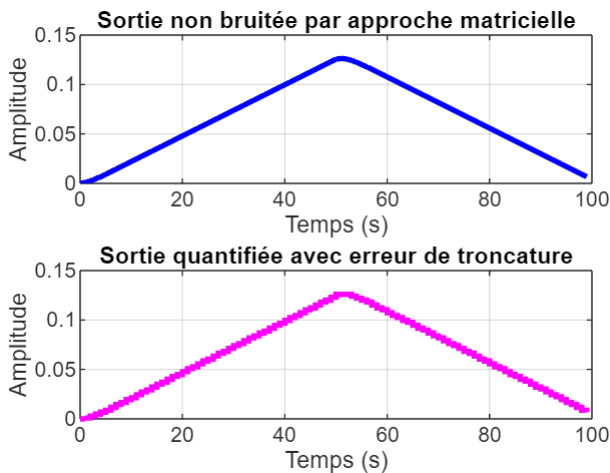
```
ynb_mat = H * x(:); % Multiplication matricielle  
  
% Conversion en vecteur ligne pour affichage  
ynb_mat = ynb_mat';
```

Application de l'erreur de troncature (quantification)

```
y_quantifie = round(ynb_mat * 2^10) / 2^10;
```

Affichage des résultats

```
figure;  
set(gcf, 'Position', [100, 100, 800, 600]);  
subplot(2,1,1);  
plot(tx, ynb_mat, 'b', 'LineWidth', 2);  
title('Sortie non bruitée par approche matricielle');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;  
  
subplot(2,1,2);  
stairs(tx, y_quantifie, 'm', 'LineWidth', 2);  
title('Sortie quantifiée avec erreur de troncature');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;
```



2. Déconvolution

a. Déconvolution « naïve » et constat de l'instabilité

11. Reconstruire le signal d'entrée en utilisant l'opération de déconvolution de Matlab (deconv),

```
% en se plaçant, dans un premier temps, dans le cas idéal où l'on a accès à la
sortie non bruitée ynb.
[x_reconstructed, r] = deconv(ynb, h); % Déconvolution naïve

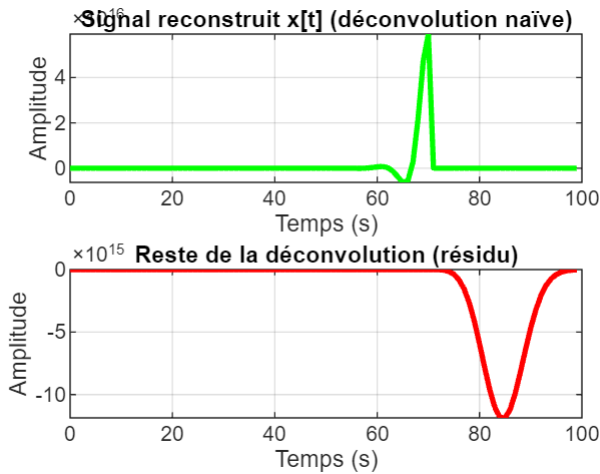
% Ajuster les tailles des vecteurs pour qu'ils correspondent à tx
L_reconstructed = length(x_reconstructed);
L_residu = length(r);

% Si la longueur du signal reconstruit est plus courte, on ajuste à la taille de tx
if L_reconstructed < Nx
    x_reconstructed = [x_reconstructed, zeros(1, Nx - L_reconstructed)];
elseif L_reconstructed > Nx
    x_reconstructed = x_reconstructed(1:Nx);
end

% Si la longueur du résidu est plus courte, on ajuste à la taille de tx
if L_residu < Nx
    r = [r, zeros(1, Nx - L_residu)];
elseif L_residu > Nx
    r = r(1:Nx);
end

% Affichage du signal reconstruit
figure;
subplot(2,1,1);
plot(tx, x_reconstructed, 'g', 'LineWidth', 2);
title('Signal reconstruit x[t] (déconvolution naïve)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

% Affichage du reste de la déconvolution (résidu)
subplot(2,1,2);
plot(tx, r, 'r', 'LineWidth', 2);
title('Reste de la déconvolution (résidu)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```



12. Pour reconstruire le signal d'entrée x_{rec} à partir des échantillons mesurés du signal de sortie y (qui est bruité), nous allons utiliser la même approche que pour la déconvolution naïve, mais cette fois-ci sur le signal bruité. L'idée est d'utiliser la méthode de déconvolution, mais sur y au lieu de y_{nb} (le signal non bruité).

```
% Utilisation de la déconvolution sur le signal bruité y
[xrec, r] = deconv(y, h); % Déconvolution naïve avec le signal bruité

% Ajuster les tailles des vecteurs pour qu'ils correspondent à tx
L_reconstructed = length(xrec);
L_residu = length(r);

% Si la longueur du signal reconstruit est plus courte, on ajuste à la taille de tx
if L_reconstructed < Nx
    xrec = [xrec, zeros(1, Nx - L_reconstructed)];
elseif L_reconstructed > Nx
    xrec = xrec(1:Nx);
end

% Si la longueur du résidu est plus courte, on ajuste à la taille de tx
if L_residu < Nx
    r = [r, zeros(1, Nx - L_residu)];
elseif L_residu > Nx
    r = r(1:Nx);
end
```

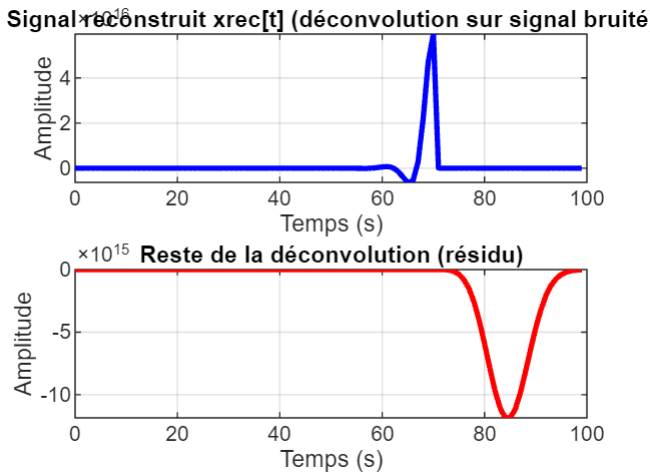


```

% Affichage du signal reconstruit
figure;
subplot(2,1,1);
plot(tx, xrec, 'b', 'LineWidth', 2);
title('Signal reconstruit xrec[t] (déconvolution sur signal bruité)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

% Affichage du reste de la déconvolution (résidu)
subplot(2,1,2);
plot(tx, r, 'r', 'LineWidth', 2);
title('Reste de la déconvolution (résidu)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

```



13- l'influence de la « forme » de la réponse impulsionnelle sur la qualité de la reconstruction en modifiant la valeur du paramètre σ .

```

sigma_values = [2, 4, 8, 12]; % Différentes valeurs de sigma à tester
figure;

for i = 1:length(sigma_values)
    sigma = sigma_values(i);
    h = (1 / (sigma * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 * sigma^2)); %
    Nouvelle impulsion

    % Convolution avec la nouvelle réponse impulsionnelle
    ynb_test = conv(x, h, 'same');

```

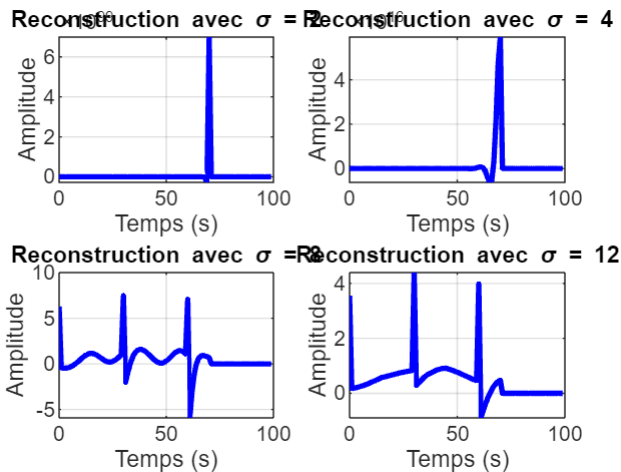
```

% Déconvolution
[x_reconstructed, ~] = deconv(ynb_test, h);

% Ajuster la taille du signal reconstruit
L_reconstructed = length(x_reconstructed);
if L_reconstructed < Nx
    x_reconstructed = [x_reconstructed, zeros(1, Nx - L_reconstructed)];
elseif L_reconstructed > Nx
    x_reconstructed = x_reconstructed(1:Nx);
end

% Affichage
subplot(2, 2, i);
plot(tx, x_reconstructed, 'b', 'LineWidth', 2);
title(['Reconstruction avec \sigma = ', num2str(sigma)]);
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
end

```



Analyse des résultats

Avec σ petit (2)

La réponse impulsionnelle est plus fine → Moins de lissage

La reconstruction est plus fidèle aux détails initiaux

Avec σ intermédiaire (4, 8)

Le signal reconstruit ressemble mieux au signal initial sans trop d'erreurs

Avec σ grand (12)

La réponse impulsionnelle est très large → Fort lissage

Le signal reconstruit est moins fidèle

b. Déconvolution par moindres carrés (ℓ_2)

```
% Définition de la réponse impulsionnelle
sigma = 4;
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 * sigma^2));
h = h / sum(h); % Normalisation

% Convolution pour obtenir le signal observé
y = conv(x, h, 'same');

% Construction de la matrice de convolution H
N = length(x);
M = length(h);
H = zeros(N, N);

for i = 1:N
    for j = 1:M
        if (i - j + 1 > 0)
            H(i, i - j + 1) = h(j);
        end
    end
end

% Déconvolution par moindres carrés
x_LS = (H' .* H) \ (H' .* y);

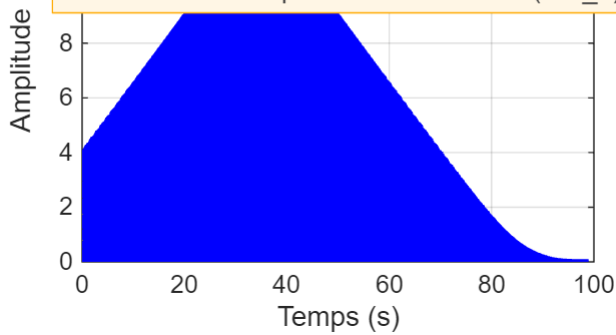
% Affichage des résultats
figure;
plot(tx, x, 'k', 'LineWidth', 2); hold on;
plot(tx, y, 'r', 'LineWidth', 1.5);
plot(tx, x_LS, 'b', 'LineWidth', 2);
legend('Signal original', 'Signal convolué', 'Signal déconvolué LS');
xlabel('Temps (s)');
ylabel('Amplitude');
title('Déconvolution par Moindres Carrés ( $\ell_2$ )');
grid on;
```

Déconvolution par Moindres Carrés (ℓ_2)

Warning: Error updating Text.



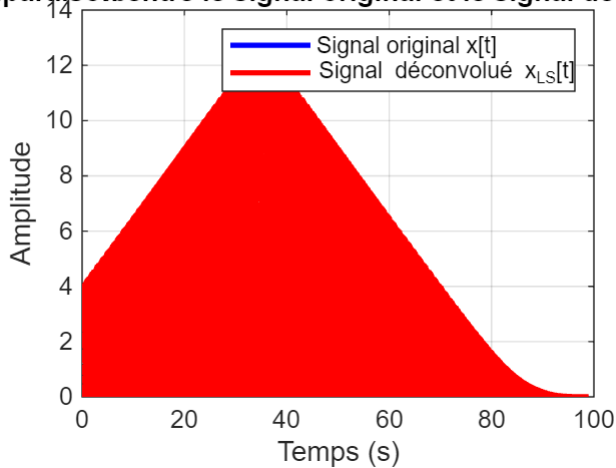
String scalar or character vector must have val
Déconvolution par Moindres Carrés (ℓ_2)



15. le signal déconvolué.

```
% Tracer le signal déconvolué
figure;
plot(tx, x, 'b', 'LineWidth', 2); % Signal d'entrée original
hold on;
plot(tx, x_LS, 'r', 'LineWidth', 2); % Signal déconvolué
legend('Signal original x[t]', 'Signal déconvolué x_{LS}[t]');
title('Comparaison entre le signal original et le signal déconvolué');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```

Comparaison entre le signal original et le signal déconvolué



```
% Calcul de l'énergie de l'erreur de reconstruction
erreur = x - x_LS;
E_erreur = sum(erreur.^ 2);

% Afficher l'énergie de l'erreur
disp(['Énergie de l'erreur de reconstruction : ', num2str(E_erreur)]);
```

Énergie de l'erreur de reconstruction : 129584.7673284369

1674192.561466134

16042630.47094788

126246773.030671

Énergie de l'erreur de reconstruction : 129584.7673284369

1674192.561466134

16042630.47094788

126246773.030671

16. Tracer les valeurs singulières de la matrice et calculer le conditionnement

```
% Calcul des valeurs singulières de la matrice H
```

```
[U, S, V] = svd(H);
```

```
% Extraire les valeurs singulières
```

```
sing_values = diag(S);
```

```
% Tracer les valeurs singulières
```

```
figure;
```

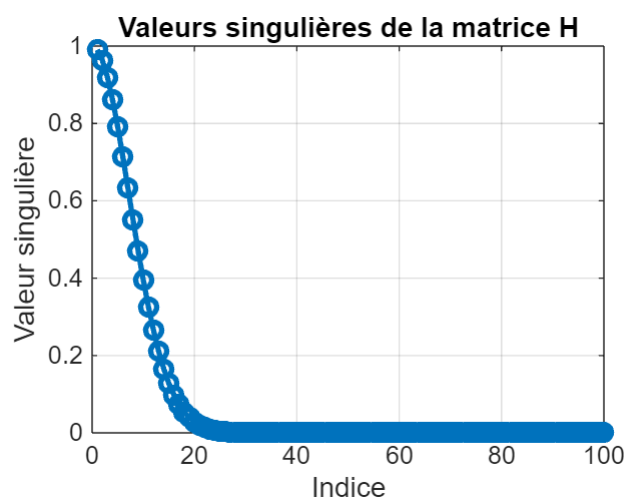
```
plot(sing_values, 'o-', 'LineWidth', 2);
```

```
title('Valeurs singulières de la matrice H');
```

```
xlabel('Indice');
```

```
ylabel('Valeur singulière');
```

```
grid on;
```



```
% Calcul du conditionnement de la matrice H (rapport entre la plus grande et la plus petite valeur singulière non nulle)
```

```
cond_H = sing_values(1) / sing_values(end);
```

```
disp(['Conditionnement de H : ', num2str(cond_H)]);
```

Conditionnement de H : 1.757089444988204e+16

17. Évolution du conditionnement en fonction de la valeur du paramètre σ du filtre

```
sigma_values = [0.1, 1, 10, 100, 1000]; % Différentes valeurs de sigma à tester
```

```
conditionnements = zeros(length(sigma_values), 1);
```

```
% Boucle pour tester différentes valeurs de sigma
```

```

for i = 1:length(sigma_values)
    sigma = sigma_values(i);

    % Créer le filtre avec une nouvelle valeur de sigma
    h = exp(-tx.^2 / (2 * sigma^2)); % Réponse impulsionnelle modifiée
    H = convmtx(h, N); % Matrice de convolution associée au filtre

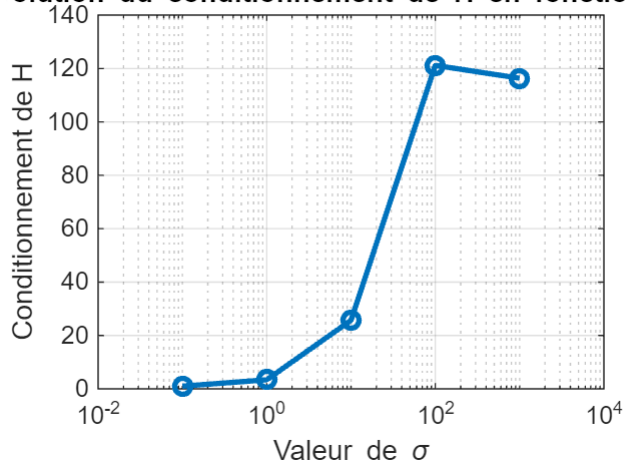
    % Calcul des valeurs singulières
    [U, S, V] = svd(H);
    sing_values = diag(S);

    % Calcul du conditionnement
    cond_H = sing_values(1) / sing_values(end);
    conditionnements(i) = cond_H;
end

% Tracer l'évolution du conditionnement en fonction de sigma
figure;
semilogx(sigma_values, conditionnements, 'o-', 'LineWidth', 2);
title('Évolution du conditionnement de H en fonction de \sigma');
xlabel('Valeur de \sigma');
ylabel('Conditionnement de H');
grid on;

```

Évolution du conditionnement de H en fonction de σ



```

options = struct('format', 'pdf', 'outputDir', 'mon_dossier');
publish('mon_script.mlx', options);

export('optimisation.mlx', 'optimisation.pdf');

```